

Інтеграція K8S з gitlab



Додавання та видалення кластерів Kubernetes



Кластери Kubernetes

GitLab пропонує інтегроване створення кластерів для таких постачальників Kubernetes:

- Google Kubernetes Engine (GKE).
- Amazon Elastic Kubernetes Service (EKS).

GitLab також може інтегруватися з будь-яким стандартним постачальником Kubernetes, як локальним, так і розміщеним.

Перш ніж почати

Перш ніж додавати кластер Kubernetes за допомогою GitLab, вам потрібно:

- Сам GitLab. Або:
 - Обліковий запис GitLab.com.
 - Самостійна інсталяція з GitLab версії 12.5 або новішої. Це гарантує, що UI GitLab можна використовувати для створення кластерів.
- Наступний доступ до GitLab:
 - Доступ супроводжувача до проекту для кластера на рівні проекту.
 - Доступ супроводжуючого до групи для кластера на рівні групи.
 - Доступ до адміністративної зони для самокерованого кластера на рівні екземпляра.

Контроль доступу

Під час створення кластера в GitLab вас запитують, чи хочете ви створити:

- Кластер керування доступом на основі ролей (RBAC), який є опцією GitLab за замовчуванням і рекомендованою опцією.
- Кластер керування доступом на основі атрибутів (ABAC).

GitLab створює необхідні сервісні облікові записи та привілеї для встановлення та запуску програм, керованих GitLab. Коли GitLab створює кластер, обліковий запис служби gitlab з правами адміністратора кластера створюється в просторі імен за замовчуванням для керування новоствореним кластером.

Коли ви вперше встановлюєте програму у свій кластер, обліковий запис служби tiller створюється з правами адміністратора кластера в просторі імен gitlab-managed-apps.

Ресурси, створені GitLab, відрізняються залежно від типу кластера.

Важливі зауваження

Зверніть увагу на наступне щодо контролю доступу:

- Спеціальні ресурси середовища створюються, лише якщо вашим кластером керує GitLab.
- Якщо ваш кластер було створено до GitLab 12.2, він використовує єдиний простір імен для всіх середовищ проекту.

Ресурси кластера RBAC

GitLab створює такі ресурси для кластерів RBAC.

Name	Type	Details	Created when
gitlab	ServiceAccount	default namespace	Creating a new cluster
gitlab-admin	ClusterRoleBinding	cluster-admin ↗ roleRef	Creating a new cluster
gitlab-token	Secret	Token for gitlab ServiceAccount	Creating a new cluster
tiller	ServiceAccount	gitlab-managed-apps namespace	Installing Helm charts
tiller-admin	ClusterRoleBinding	cluster-admin ↗ roleRef	Installing Helm charts
Environment namespace	Namespace	Contains all environment-specific resources	Deploying to a cluster
Environment namespace	ServiceAccount	Uses namespace of environment	Deploying to a cluster
Environment namespace	Secret	Token for environment ServiceAccount	Deploying to a cluster
Environment namespace	RoleBinding	admin ↗ roleRef	Deploying to a cluster

Ресурси кластера ABAC

GitLab створює такі ресурси для кластерів ABAC

Name	Type	Details	Created when
gitlab	ServiceAccount	default namespace	Creating a new cluster
gitlab-token	Secret	Token for gitlab ServiceAccount	Creating a new cluster
tiller	ServiceAccount	gitlab-managed-apps namespace	Installing Helm charts
tiller-admin	ClusterRoleBinding	cluster-admin roleRef	Installing Helm charts
Environment namespace	Namespace	Contains all environment-specific resources	Deploying to a cluster
Environment namespace	ServiceAccount	Uses namespace of environment	Deploying to a cluster
Environment namespace	Secret	Token for environment ServiceAccount	Deploying to a cluster

Безпека виконавців

За замовчуванням для виконавців увімкнено привілейований режим, який дозволяє їм виконувати спеціальні команди та запускати Docker у Docker. Ця функція потрібна для запуску деяких завдань Auto DevOps. Це означає, що контейнери працюють у привілейованому режимі, тому ви повинні знати про деякі важливі деталі.

Привілейований прапор надає всі можливості запущеному контейнеру, який, у свою чергу, може робити майже все, що може робити хост. Майте на увазі невід'ємний ризик безпеки, пов'язаний із виконанням операцій докерів над довільними зображеннями, оскільки вони фактично мають кореневий доступ.

Якщо ви не хочете використовувати runners у привілейованому режимі:

- Використовуйте спільні runners на GitLab.com. Вони не мають цієї проблеми безпеки.
- Налаштуйте власні runners за допомогою конфігурації, описаної на спільних runners. Це передбачає:
 1. Переконайтеся, що він не встановлений через програми.
 2. Встановіть runner за допомогою docker+machine.

Створіть новий кластер

Нові кластери можна створювати за допомогою GitLab на Google Kubernetes Engine (GKE) або Amazon Elastic Kubernetes Service (EKS) на рівні проекту, групи чи екземпляра:

1. Перейдіть до свого:
 - Сторінка «Операції проекту» > Kubernetes для кластера на рівні проекту.
 - Сторінка Kubernetes групи для кластера на рівні групи.
 - Область адміністратора > сторінка Kubernetes для кластера на рівні екземпляра.
2. Натисніть Додати кластер Kubernetes.
3. Натисніть вкладку Створити новий кластер.
4. Натисніть Amazon EKS або Google GKE і дотримуйтеся вказівок для потрібної служби:
 - Amazon EKS.
 - Google GKE.

Додати існуючий кластер

Якщо у вас є кластер Kubernetes, ви можете додати його до проекту, групи чи екземпляра.

Інтеграція Kubernetes не підтримується для кластерів arm64. Після додавання існуючого кластера ви можете встановити для нього runners, як описано в GitLab Managed Apps.

Наявний кластер Kubernetes

Щоб додати кластер Kubernetes до проекту, групи чи екземпляра:

1. Перейдіть до свого:
 - a. Сторінка «**Операції проекту**» > **Kubernetes** для кластера на рівні проекту.
 - b. Сторінка **Kubernetes** групи для кластера на рівні групи.
 - c. **Область адміністратора** > сторінка **Kubernetes** для кластера на рівні екземпляра.

Додати існуючий кластер

2. Натисніть Додати кластер Kubernetes.
3. Клацніть вкладку Додати існуючий кластер і введіть деталі:
 - a. Ім'я кластера Kubernetes (обов'язково) – ім'я, яке ви хочете дати кластеру.
 - b. Обсяг середовища (обов'язково)
 - c. URL-адреса API (обов'язково) – це URL-адреса, яку GitLab використовує для доступу до API Kubernetes. Kubernetes надає кілька API, нам потрібна «базова» URL-адреса, спільна для всіх них. Наприклад, <https://kubernetes.example.com>, а не <https://kubernetes.example.com/api/v1>.
 - d. Отримайте URL-адресу API, виконавши цю команду:

```
kubect1 cluster-info | grep -E 'Kubernetes master|Kubernetes control plane' | awk '/http/ {print $NF}'
```

- e. Сертифікат ЦС (обов'язково) – для автентифікації в кластері потрібен дійсний сертифікат Kubernetes. Ми використовуємо сертифікат, створений за замовчуванням.
 - i. Перелічіть секрети за допомогою `kubect1 get secrets`, і один має мати назву, схожу на `default-token-xxxxx`. Скопіюйте цю назву маркера для використання нижче.
 - ii. Отримайте сертифікат, виконавши цю команду:

```
kubect1 get secret <secret name> -o jsonpath="{['data']['ca.crt']}" | base64 --decode
```

Додати існуючий кластер

Якщо команда повертає весь ланцюжок сертифікатів, ви повинні скопіювати кореневий сертифікат ЦС і будь-які проміжні сертифікати в кінці ланцюжка. Файл ланцюга має на

```
-----BEGIN MY CERTIFICATE-----  
-----END MY CERTIFICATE-----  
-----BEGIN INTERMEDIATE CERTIFICATE-----  
-----END INTERMEDIATE CERTIFICATE-----  
-----BEGIN INTERMEDIATE CERTIFICATE-----  
-----END INTERMEDIATE CERTIFICATE-----  
-----BEGIN ROOT CERTIFICATE-----  
-----END ROOT CERTIFICATE-----
```

е. Маркер — GitLab автентифікує Kubernetes за допомогою службових маркерів, які мають певний простір імен. Використаний маркер має належати обліковому запису служби з правами адміністратора кластера. Щоб створити обліковий запис цієї служби:

- i. Створіть файл gitlab-admin-service-account.yaml із вмістом:

```
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: gitlab  
  namespace: kube-system  
---  
apiVersion: rbac.authorization.k8s.io/v1beta1  
kind: ClusterRoleBinding  
metadata:  
  name: gitlab-admin  
roleRef:  
  apiGroup: rbac.authorization.k8s.io  
  kind: ClusterRole  
  name: cluster-admin  
subjects:  
- kind: ServiceAccount  
  name: gitlab  
  namespace: kube-system
```

Додати існуючий кластер

- ii. Застосуйте прив'язку облікового запису служби та ролі кластера до свого кластера:

```
kubectl apply -f gitlab-admin-service-account.yaml
```

Для створення ролей на рівні кластера вам потрібен дозвіл `container.clusterRoleBindings/create`. Якщо у вас немає цього дозволу, ви також можете ввімкнути основну автентифікацію, а потім запустити команду `kubectl apply` як адміністратор:

```
kubectl apply -f gitlab-admin-service-account.yaml --username=admin --password=<password>
```

Вихід:

```
serviceaccount "gitlab" created  
clusterrolebinding "gitlab-admin" created
```

Додати існуючий кластер

- iii. Отримайте маркер для облікового запису служби gitlab:

```
kubectl -n kube-system describe secret $(kubectl -n kube-system get secret | grep gitlab | awk '{print $1}')
```

Скопіюйте значення <authentication token> із результату:

```
Name:          gitlab-token-b5zv4
Namespace:     kube-system
Labels:        <none>
Annotations:   kubernetes.io/service-account.name=gitlab
               kubernetes.io/service-account.uid=bcfe66ac-39be-11e8-97e8-026dce96b6e8

Type: kubernetes.io/service-account-token

Data
====
ca.crt:      1025 bytes
namespace:   11 bytes
token:       <authentication_token>
```

Додати існуючий кластер

- f. Кластер, керований GitLab. Залиште цей прапорець, якщо ви хочете, щоб GitLab керував просторами імен і обліковими записами служб для цього кластера.
 - g. Простір імен проекту (необов'язково) – Вам не потрібно його заповнювати; залишивши його порожнім, GitLab створить його для вас. Також:
 - Кожен проект повинен мати унікальний простір імен.
 - Простір імен проекту не обов'язково є простором імен секрету, якщо ви використовуєте секрет із ширшими дозволами, наприклад секрет за замовчуванням.
 - Ви не повинні використовувати default як простір імен проекту.
 - Якщо ви або хтось створив секрет спеціально для проекту, як правило, з обмеженими дозволами, простір імен секрету та простір імен проекту можуть бути однаковими.
4. Нарешті натисніть кнопку **«Створити кластер Kubernetes»**.

Через пару хвилин ваш кластер готовий. Тепер ви можете перейти до встановлення деяких попередньо визначених програм.

Вимкнути керування доступом на основі ролей (RBAC) (необов'язково)

Підключаючи кластер через інтеграцію GitLab, ви можете вказати, чи підтримує кластер RBAC чи ні. Це впливає на те, як GitLab взаємодіє з кластером для певних операцій. Якщо під час створення ви не встановили прапорець кластера з увімкненим RBAC, GitLab припускає, що RBAC вимкнено для вашого кластера під час взаємодії з ним. Якщо так, ви повинні вимкнути RBAC у своєму кластері, щоб інтеграція працювала належним чином.

☒ **RBAC-enabled cluster**

Enable this setting if using role-based access control (RBAC). This option will allow you to install applications on RBAC clusters.

Щоб ефективно вимкнути RBAC, можна застосувати глобальні дозволи, які надають повний доступ:

```
kubectl create clusterrolebinding permissive-binding \
  --clusterrole=cluster-admin \
  --user=admin \
  --user=kubelet \
  --group=system:serviceaccounts
```

Увімкнення або вимкнення інтеграції

Інтеграція кластера Kubernetes активується після того, як ви успішно створили новий кластер або додали існуючий. Щоб вимкнути інтеграцію кластера Kubernetes:

1. Перейдіть до свого:
 - Сторінка «**Операції проекту**» > **Kubernetes** для кластера на рівні проекту.
 - Сторінка **Kubernetes** групи для кластера на рівні групи.
 - **Область адміністратора** > сторінка **Kubernetes** для кластера на рівні екземпляра.
2. Натисніть на назву кластера.
3. Натисніть перемикач **GitLab Integration**.
4. Натисніть **Зберегти зміни**

Видалення інтеграції

Щоб видалити інтеграцію кластера Kubernetes зі свого проекту, спершу перейдіть на вкладку «**Додаткові налаштування**» на сторінці деталей кластера та виконайте одну з наведених нижче дій.

- Виберіть **Вилучити інтеграцію**, щоб видалити лише інтеграцію Kubernetes.
- У GitLab 12.6 виберіть «**Видалити інтеграцію та ресурси**», щоб також видалити всі пов'язані ресурси кластера GitLab (наприклад, простори імен, ролі та прив'язки) під час видалення інтеграції.

При видаленні інтеграції кластера зверніть увагу:

- Щоб видалити інтеграцію кластера Kubernetes, вам потрібні дозволи супроводжувача або вище.
- Коли ви видаляєте кластер, ви видаляєте лише його зв'язок із GitLab, а не сам кластер. Щоб видалити кластер, ви можете зробити це, відвідавши інформаційну панель GKE або EKS або використовуючи `kubect1`.



Кластери Kubernetes

Налаштування

Підтримувані версії кластера

GitLab зобов'язується підтримувати принаймні дві готові до виробництва проміжні версії Kubernetes у будь-який момент часу. Ми регулярно переглядаємо версії, які підтримуємо, і надаємо тримісячний період припинення підтримки, перш ніж припинити підтримку певної версії. Діапазон підтримуваних версій базується на оцінці:

- Версії, які підтримують основні керовані постачальники Kubernetes.
- Версії, які підтримуються спільнотою Kubernetes.

Налаштування

GitLab підтримує такі версії Kubernetes, і ви можете будь-коли оновити свою версію Kubernetes до будь-якої підтримуваної:

- 1.19 (підтримка закінчується 22 лютого 2022 р.)
- 1.18 (підтримка закінчується 22 листопада 2021 р.)
- 1.17 (підтримка закінчується 22 вересня 2021 р.)
- 1.16 (підтримка закінчується 22 липня 2021 р.)
- 1.15 (підтримка закінчується 22 травня 2021 р.)
- 1.14 (не підтримується, підтримка закінчується 22 грудня 2020 року)

Деякі функції GitLab можуть підтримувати версії за межами наведеного тут діапазону.

Налаштування

Кілька кластерів Kubernetes

Ви можете пов'язати з проектом більше одного кластера Kubernetes. Таким чином ви можете мати різні кластери для різних середовищ, як-от розробка, постановка, виробництво тощо. Додайте ще один кластер, як і вперше, і переконайтеся, що встановили область середовища, яка відрізнятиме новий кластер від решти.

Встановлення області дії середовища

Додаючи більше одного кластера Kubernetes до свого проекту, вам потрібно розрізнити їх за допомогою середовища. Область середовища пов'язує кластери з середовищами, подібними до того, як працюють змінні середовища.

Область середовища за замовчуванням — *, що означає, що всі завдання, незалежно від середовища, використовують цей кластер. Кожна область може використовуватися лише одним кластером у проекті, і в іншому випадку виникає помилка перевірки. Крім того, завдання, для яких не встановлено ключове слово середовища, не можуть отримати доступ до жодного кластера.

Налаштування

Наприклад, припустимо, що в проекті існують такі кластери Kubernetes

Cluster	Environment scope
Development	*
Production	production

І встановлюються такі середовища `.gitlab-ci.yml`:

```
stages:
  - test
  - deploy

test:
  stage: test
  script: sh test

deploy to staging:
  stage: deploy
  script: make deploy
  environment:
    name: staging
    url: https://staging.example.com/

deploy to production:
  stage: deploy
  script: make deploy
  environment:
    name: production
    url: https://example.com/
```

Результати:

- Деталі кластера розробки доступні в розгортанні до проміжного завдання.
- Деталі виробничого кластера доступні в завданні розгортання до виробництва.
- У тестовому завданні немає деталей кластера, оскільки воно не визначає жодного середовища.

Налаштування кластера Kubernetes

Після додавання кластера Kubernetes до GitLab прочитайте цей розділ, у якому описано важливі аспекти налаштування кластерів Kubernetes за допомогою GitLab.

Наслідки безпеки

Конфігурація кластера за замовчуванням надає доступ до широкого набору функцій, необхідних для успішного створення та розгортання контейнерної програми. Майте на увазі, що однакові облікові дані використовуються для всіх програм, запущених у кластері.

Керовані GitLab кластери

Ви можете дозволити GitLab керувати вашим кластером за вас. Якщо вашим кластером керує GitLab, ресурси для ваших проектів створюються автоматично. Якщо ви вирішите керувати власним кластером, ресурси для конкретного проекту не створюватимуться автоматично. Якщо ви використовуєте Auto DevOps, ви повинні явно вказати змінну розгортання `KUBE_NAMESPACE` для використання в завданнях розгортання. В іншому випадку для вас буде створено простір імен.

Налаштування кластера Kubernetes

Важливі зауваження

Зверніть увагу на наступне з GitLab і кластерами:

- Якщо ви встановлюєте програми на своєму кластері, GitLab створить ресурси, необхідні для їх запуску, навіть якщо ви вирішили керувати власним кластером.
- Майте на увазі, що ручне керування ресурсами, створеними GitLab, наприклад просторами імен і обліковими записами служб, може спричинити несподівані помилки. Якщо це станеться, спробуйте очистити кеш кластера.

Налаштування кластера Kubernetes

Очищення кешу кластера

Представлено в GitLab 12.6.

Якщо ви дозволяєте GitLab керувати вашим кластером, GitLab зберігає кешовану версію просторів імен і облікових записів служб, створених для ваших проектів. Якщо ви зміните ці ресурси у своєму кластері вручну, цей кеш може вийти з синхронізації з вашим кластером. Це може призвести до збою завдань розгортання.

Щоб очистити кеш:

1. Перейдіть на сторінку **Operations > Kubernetes** вашого проекту та виберіть свій кластер.
2. Розгорніть розділ «**Додаткові налаштування**».
3. Натисніть **Очистити кеш кластера**.

Базовий домен

Представлено в GitLab 11.8.

Під час використання GitLab Serverless вам не потрібно вказувати базовий домен у налаштуваннях кластера. У цьому випадку домен вказується як частина інсталяції Knative.

Зазначення базового домену автоматично встановлює `KUBE_INGRESS_BASE_DOMAIN` як змінну середовища. Якщо ви використовуєте Auto DevOps, цей домен використовується для різних етапів. Наприклад, Auto Review Apps і Auto Deploy.

Домен повинен мати символ підстановки DNS, налаштований на IP-адресу Ingress. Після встановлення Ingress ви можете:

- Створіть запис A, який вказує на IP-адресу Ingress у постачальника домену.
- Введіть адресу DNS із символом підстановки за допомогою служби, наприклад `nip.io` або `xip.io`.
Наприклад, `192.168.1.1.xip.io`.

Базовий домен

Для визначення зовнішньої IP-адреси Ingress або зовнішнього імені хоста Ingress:

- Якщо кластер знаходиться на GKE:
 1. Клацніть посилання **Google Kubernetes Engine** у **Розширених налаштуваннях** або перейдіть безпосередньо до інформаційної панелі Google Kubernetes Engine.
 2. Виберіть правильний проект і кластер.
 3. Натисніть **Connect**
 4. Виконайте команду `gcloud` у локальному терміналі або за допомогою **Cloud Shell**.
- *Якщо кластер не на GKE:* дотримуйтеся конкретних інструкцій для вашого постачальника Kubernetes, щоб налаштувати `kubectl` за допомогою правильних облікових даних. Результати наведених нижче прикладів показують зовнішню кінцеву точку вашого кластера. Потім цю інформацію можна використовувати для налаштування записів DNS і правил пересилання, які дозволяють зовнішній доступ до ваших розгорнутих програм.

Базовий домен

Залежно від вашого Ingress зовнішню IP-адресу можна отримати різними способами. У цьому списку представлено загальне рішення та деякі специфічні підходи GitLab:

- Загалом, ви можете отримати список IP-адрес усіх балансувальників навантаження, виконавши:

```
kubectl get svc --all-namespaces -o jsonpath='{range.items[?(@.status.loadBalancer.ingress)]}{.status.loadBalancer.ingress[*].ip} '
```

- Якщо ви встановили Ingress за допомогою програм, запустіть:

```
kubectl get service --namespace=gitlab-managed-apps ingress-nginx-ingress-controller -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

Деякі кластери Kubernetes замість цього повертають ім'я хоста, наприклад Amazon EKS. Для цих платформ запустіть:

```
kubectl get service --namespace=gitlab-managed-apps ingress-nginx-ingress-controller -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

Базовий домен

Якщо ви використовуєте EKS, також створюється Elastic Load Balancer, що спричиняє додаткові витрати на AWS.

- Istio/Knative використовує іншу команду. Запустіть:

```
kubectl get svc --namespace=istio-system istio-ingressgateway -o jsonpath='{.status.loadBalancer.ingress[0].ip} '
```

Якщо ви бачите кінцевий % у деяких версіях Kubernetes, не включайте його.

Встановлення програм

GitLab може встановлювати та керувати деякими програмами, такими як Helm, GitLab Runner, Ingress, Prometheus тощо, у вашому кластері на рівні проекту.

Auto DevOps

Auto DevOps автоматично виявляє, будує, тестує, розгортає та контролює ваші програми.

Щоб повністю використовувати Auto DevOps (автоматичне розгортання, автоматичний перегляд додатків і автоматичний моніторинг), потрібно увімкнути інтеграцію проекту Kubernetes. Однак кластери Kubernetes можна використовувати без Auto DevOps.

Розгортання в кластері Kubernetes

Кластер Kubernetes може бути місцем призначення для завдання розгортання. Якщо

- Кластер інтегровано з GitLab, для вашої роботи доступні спеціальні змінні розгортання, а конфігурація не потрібна. Ви можете негайно почати взаємодіяти з кластером зі своїх робочих місць за допомогою таких інструментів, як `kubectl` або `helm`.
- Ви не використовуєте інтеграцію кластера GitLab, ви все одно можете розгорнути його у своєму кластері. Однак ви повинні самостійно налаштувати інструменти Kubernetes за допомогою змінних середовища, перш ніж ви зможете взаємодіяти з кластером зі своїх завдань.

Змінні розгортання

Змінні розгортання потребують дійсного маркера розгортання під назвою `gitlab-deploy-token` наступної команди у вашому сценарії завдання розгортання, щоб Kubernetes міг отримати доступ до реєстру:

- Використання Kubernetes 1.18+:

```
kubectl create secret docker-registry gitlab-registry --docker-server="$SCI_REGISTRY"  
--docker-username="$SCI_DEPLOY_USER" --docker-password="$SCI_DEPLOY_PASSWORD"  
--docker-email="$GITLAB_USER_EMAIL" -o yaml --dry-run=client | kubectl apply -f -
```

- Використання Kubernetes <1.18:

```
kubectl create secret docker-registry gitlab-registry --docker-server="$SCI_REGISTRY"  
--docker-username="$SCI_DEPLOY_USER" --docker-password="$SCI_DEPLOY_PASSWORD"  
--docker-email="$GITLAB_USER_EMAIL" -o yaml --dry-run | kubectl apply -f -
```

Змінні розгортання

Інтеграція кластера Kubernetes надає ці змінні розгортання в середовищі збірки GitLab CI/CD для завдань розгортання. Завдання розгортання визначили цільове середовище.

Variable	Description
<code>KUBE_URL</code>	Equal to the API URL.
<code>KUBE_TOKEN</code>	The Kubernetes token of the environment service account . Prior to GitLab 11.5, <code>KUBE_TOKEN</code> was the Kubernetes token of the main service account of the cluster integration.
<code>KUBE_NAMESPACE</code>	The namespace associated with the project's deployment service account. In the format <code><project_name>-<project_id>-<environment></code> . For GitLab-managed clusters, a matching namespace is automatically created by GitLab in the cluster. If your cluster was created before GitLab 12.2, the default <code>KUBE_NAMESPACE</code> is set to <code><project_name>-<project_id></code> .
<code>KUBE_CA_PEM_FILE</code>	Path to a file containing PEM data. Only present if a custom CA bundle was specified.
<code>KUBE_CA_PEM</code>	(deprecated) Raw PEM data. Only if a custom CA bundle was specified.
<code>KUBECONFIG</code>	Path to a file containing <code>kubeconfig</code> for this deployment. CA bundle would be embedded if specified. This configuration also embeds the same token defined in <code>KUBE_TOKEN</code> so you likely need only this variable. This variable name is also automatically picked up by <code>kubect1</code> so you don't need to reference it explicitly if using <code>kubect1</code> .
<code>KUBE_INGRESS_BASE_DOMAIN</code>	From GitLab 11.8, this variable can be used to set a domain per cluster. See cluster domains for more information.

Спеціальний простір імен

Інтеграція Kubernetes надає `KUBECONFIG` із автоматично створеним простором імен для завдань розгортання. За умовчанням використовуються простори імен, специфічні для середовища проекту, у формі `<prefix>-<environment>`, де `<prefix>` має форму `<project_name>-<project_id>`

Ви можете налаштувати простір імен розгортання кількома способами:

- Ви можете вибрати **простір імен для середовища** або **простір імен для проекту**. Простір імен для середовища є стандартним і рекомендованим параметром, оскільки він запобігає змішуванню ресурсів між робочим і невиробничим середовищами.
- При використанні кластера на рівні проекту ви можете додатково налаштувати префікс простору імен. Якщо використовується простір імен для середовища, простір імен розгортання є `<prefix>-<environment>` але в інших випадках просто `<prefix>`.
- Для **некерованих** кластерів автоматично згенерований простір імен встановлюється в `KUBECONFIG`, але користувач відповідає за його існування. Ви можете повністю налаштувати це значення за допомогою середовища `kubernetes:namespacein .gitlab-ci.yml`.

Інтеграції

Розгортання Canary

Використовуйте розгортання Kubernetes Canary і візуалізуйте свої розгортання Canary прямо на панелі розгортання, не виходячи з GitLab.

Розгорніть дошки

Плати розгортання GitLab пропонують консолідоване уявлення про поточну працездатність і стан кожного середовища CI, що працює на Kubernetes. Вони відображають статус модулів у розгортанні. Розробники та інші члени команди можуть переглядати хід і статус розгортання пакет за пакетом у робочому процесі, який вони вже використовують, без доступу до Kubernetes.

Перегляд журналів контейнерів

GitLab дозволяє переглядати журнали запущених модулів у підключених кластерах Kubernetes. Відображаючи журнали безпосередньо в GitLab, розробники можуть уникнути необхідності керувати інструментами консолі або переходити до іншого інтерфейсу.

Інтеграції

Веб-термінали

Представлено в GitLab 8.15.

Якщо ввімкнено, інтеграція Kubernetes додає підтримку веб-терміналів у ваше середовище. Це засновано на функції `exec`, наявній у Docker і Kubernetes, тож ви отримуєте новий сеанс оболонки у своїх існуючих контейнерах. Щоб скористатися цією інтеграцією, вам слід розгорнути Kubernetes за допомогою наведених вище змінних розгортання, переконавшись, що будь-які розгортання, набори реплік і пакети мають анотації:

- `app.gitlab.com/env: $CI_ENVIRONMENT_SLUG`
- `app.gitlab.com/app: $CI_PROJECT_PATH_SLUG`

`$CI_ENVIRONMENT_SLUG` та `$CI_PROJECT_PATH_SLUG` є значеннями змінних CI.

Щоб використовувати термінали, ви повинні бути власником проекту або мати дозвіл супроводжувачого. Підтримка обмежена першим контейнером у першому пакеті вашого середовища.

Усунення несправностей

Перед початком завдань розгортання GitLab створює наступне спеціально для завдання розгортання:

- Простір імен.
- Сервісний обліковий запис.

Однак іноді GitLab не може їх створити. У таких випадках ваше завдання може бути невдалим із повідомленням:

```
This job failed because the necessary resources were not successfully created.
```

Щоб знайти причину цієї помилки під час створення простору імен і облікового запису служби, перевірте журнали.

Причини невдачі:

- Маркер, який ви надали GitLab, не має привілеїв адміністратора кластера необхідних для GitLab.
- Відсутні змінні `KUBECONFIG` або `KUBE_TOKEN`. Щоб бути переданими у вашу роботу, вони повинні мати відповідне середовище: ім'я. Якщо для вашого завдання не встановлено середовище: ім'я, облікові дані Kubernetes йому не передадуться.

Моніторинг вашого кластера Kubernetes

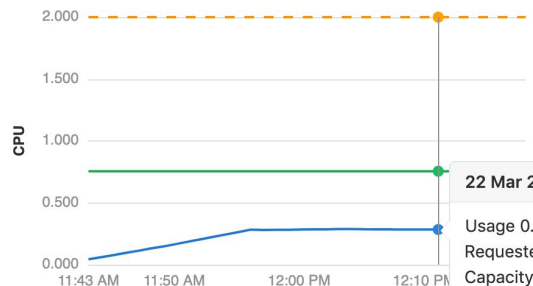
Автоматично виявляти та контролювати показники Kubernetes. Також підтримується автоматичний моніторинг NGINX Ingress.

Візуалізація працездатності кластера

Коли Prometheus розгорнуто, GitLab відстежує працездатність кластера. У верхній частині сторінки налаштувань кластера відображається використання ЦП і пам'яті, а також загальний доступний обсяг. Слідкувати за ресурсами кластера може бути важливо, якщо в кластері закінчиться пам'ять, модулі можуть бути вимкнені або

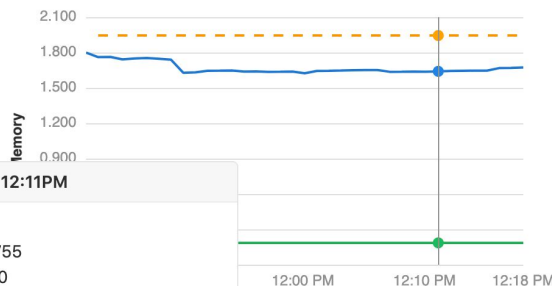
Cluster health

CPU Usage



— Usage Avg: 0.236 · Max: 0.288
— Requested Avg: 0.755 · Max: 0.755
- - Capacity Avg: 2 · Max: 2

Memory usage



— Usage Avg: 1.668 · Max: 1.799
— Requested Avg: 0.186 · Max: 0.186
- - Capacity Avg: 1.944 · Max: 1.944

Питання та відповіді